

Article

Research on Path Planning Method for Mobile Platforms Based on Hybrid Swarm Intelligence Algorithms in Multi-Dimensional Environments

Shuai Wang ¹, Yifan Zhu ², Yuhong Du ^{3,4,*}  and Ming Yang ⁵¹ School of Mechanical and Automotive Engineering, Liaocheng University, Liaocheng 252000, China; shuaiwang@lcu.edu.cn² College of Management and Economics, Tianjin University, Tianjin 300072, China; yfzu@tju.edu.cn³ Key Laboratory of Advanced Mechatronics Equipment Technology, Tiangong University, Tianjin 300387, China⁴ School of Mechanical Engineering, Tiangong University, Tianjin 300387, China⁵ Analysis & Testing Center, Beihang University, Beijing 100191, China; myang_16@buaa.edu.cn

* Correspondence: dyh202@163.com

Abstract

Traditional algorithms such as Dijkstra and APF rely on complete environmental information for path planning, which results in numerous constraints during modeling. This not only increases the complexity of the algorithms but also reduces the efficiency and reliability of the planning. Swarm intelligence algorithms possess strong data processing and search capabilities, enabling them to efficiently solve path planning problems in different environments and generate approximately optimal paths. However, swarm intelligence algorithms suffer from issues like premature convergence and a tendency to fall into local optima during the search process. Thus, an improved Artificial Bee Colony-Beetle Antennae Search (IABCBAS) algorithm is proposed. Firstly, Tent chaos and non-uniform variation are introduced into the bee algorithm to enhance population diversity and spatial searchability. Secondly, the stochastic reverse learning mechanism and greedy strategy are incorporated into the beetle antennae search algorithm to improve direction-finding ability and the capacity to escape local optima, respectively. Finally, the weights of the two algorithms are adaptively adjusted to balance global search and local refinement. Results of experiments using nine benchmark functions and four comparative algorithms show that the improved algorithm exhibits superior path point search performance and high stability in both high- and low-dimensional environments, as well as in unimodal and multimodal environments. Ablation experiment results indicate that the optimization strategies introduced in the algorithm effectively improve convergence accuracy and speed during path planning. Results of the path planning experiments show that compared with the comparison algorithms, the average path planning distance of the improved algorithm is reduced by 23.83% in the 2D multi-obstacle environment, and the average planning time is shortened by 27.97% in the 3D surface environment. The improvement in path planning efficiency makes this algorithm of certain value in engineering applications.

Keywords: optimized search algorithm; adaptive balance; search performance evaluation; path planning applications



Academic Editor: Yongquan Zhou

Received: 4 July 2025

Revised: 25 July 2025

Accepted: 30 July 2025

Published: 1 August 2025

Citation: Wang, S.; Zhu, Y.; Du, Y.; Yang, M. Research on Path Planning Method for Mobile Platforms Based on Hybrid Swarm Intelligence Algorithms in Multi-Dimensional Environments. *Biomimetics* **2025**, *10*, 503. <https://doi.org/10.3390/biomimetics10080503>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Path planning is a key technology for mobile platforms to realize intelligent functions such as environmental sensing and autonomous operation. Algorithms are utilized to

enable mobile platforms to find optimal feasible paths from the initial point to the target location in different spaces, thereby improving operational efficiency [1]. Path planning algorithms can be broadly categorized into two classes: traditional algorithms and bionic intelligence-based algorithms [2], and they are widely used in fields such as electric power inspection [3], underwater search [4], and military task assignment [5]. Traditional algorithms can be subdivided into global and local path planning algorithms according to the utilization degree of environmental information. Global path planning provides approximate paths, while local path planning makes corrections within a local range of the global path to adapt to environmental changes and special needs. The global path planning algorithm needs to consider obstacle information in the environmental map, with representative algorithms including the A* and RRT algorithms [6]; the local path planning algorithm needs to update real-time environmental information via sensors, with representative algorithms such as the artificial potential field method and D* algorithm [7]. With the development of artificial intelligence technologies, bionic swarm intelligence algorithms that simulate biological evolution, group social behavior, and information exchange in nature have been formulated using mathematical expressions [8]. Early representative algorithms of swarm intelligence algorithms are particle swarm optimization (PSO) and ant colony optimization (ACO) [9]; medium-term representative algorithms are artificial bee colony (ABC) and artificial fish swarm algorithm (AFSA) [10]; and recent representative algorithms are sparrow search algorithm (SSA), whale optimization algorithm (WOA), and beetle antennae search (BAS), etc. [11]. The swarm intelligence algorithms have independent solution thinking and they can be adjusted adaptively and iteratively according to the path planning characteristics and requirements. Rich population categories help to achieve parallel computing and improve search efficiency while also having excellent scalability for larger-scale and more complex scenarios [12].

Swarm intelligence algorithms include two phases: global exploration and local optimization. The former performs a highly stochastic global search, while the latter performs deep optimization as the number of iterations in the search area increases. Regulating the balance between these two phases is particularly important. If the weight in the early exploration stage is too large, the algorithm may generate an unordered search, which slows down the convergence speed and wastes computing resources. If the weight in the later stage is too large, the algorithm may fall into local optimal oscillations and be unable to escape extreme points in the later stage [13]. The swarm intelligence algorithms do not rely on the initial parameter settings, and they are more inclined to explore the environmental information with a wider search range. The beetle algorithm has a high degree of structuring. It can not only simulate an individual beetle to judge the information concentration in the environment and solve problems independently but also be converted into a population of beetles to establish information interaction for a common solution. Some scholars combined the powerful search ability of the swarm algorithm with the low complexity of the beetle algorithm to improve the global search ability and local planning speed of the algorithm [14]. Cui et al. [15] introduced globally optimal individuals into both the employed bees and scout bees stages and introduced the teaching- and learning-based optimization algorithm into the peak-peak stage to enhance the algorithm's mining ability and searching efficiency in terms of swarm population. Ni et al. [16] proposed a multi-strategy integrated bee colony algorithm, which utilizes different individual experiences with multiple search strategies to update the global path. Li [17] and Comert [18] combined the bee colony algorithm with the ant colony algorithm to improve the quality of the initial solution, optimize the number of path turns, and avoid blind searches. Liang et al. [19] added a virtual goal to the global search points to achieve fast local search optimization. Yu et al. [20] divided the global search process into several segments using the water flow

potential field method and optimized the local search coordinate points according to the obstacles' characteristics using the beetle algorithm to prevent falling into local optimal points. In addition, introducing chaotic functions and reverse learning mechanisms are the main strategies for improving the performance of swarm intelligence algorithms. Chen et al. [21] and Geng et al. [22] incorporated non-uniform variation and chaotic mapping functions into the sparrow search algorithm, respectively, which enhances the randomness and traversal ability of the populations. Mohapatra et al. [23] introduced a stochastic reverse learning mechanism into the golden jackal algorithm to overcome the limitations of balancing exploration and exploitation and improve the convergence speed. Hu et al. [24] introduced variational and greedy strategies into the rabbit algorithm to improve the quality of spatial solution exploration and population exploitation. The aforementioned literature has improved the efficiency of algorithms in path planning by means of hybrid swarm intelligence algorithms, introducing optimization strategies, etc. However, there are still problems such as early convergence of global exploration, low local optimization accuracy, and high dependence on parameter settings.

According to the No Free Lunch Theorem [25], no single intelligent optimization algorithm can universally solve all path planning problems. Even a well-optimized single algorithm fails to simultaneously address the dual challenges of global search and local optimization. To tackle this, the IABCBAS algorithm is tailored for path planning in multi-dimensional environments. This algorithm leverages swarm intelligence mechanisms for global exploration of path waypoints while employing the beetle algorithm for local path refinement. The main contributions of this study are summarized as follows.

Firstly, Tent chaotic perturbation is employed to enhance bee population diversity. By non-uniformly adjusting the updated swarm positions, the algorithm broadens the distribution of solution sets, optimizes search point quality, and boosts convergence speed. Secondly, a random reversal strategy is adopted to increase search direction points, thereby expanding the optimization range of the beetle algorithm and strengthening its ability to escape local optima. Position-based greedy selection is employed to achieve higher convergence accuracy. Thirdly, an adaptive inertia weight factor is introduced to autonomously adjust the weight distribution between the two algorithms, thereby achieving a balanced trade-off between global search capability and local optimization performance. Fourthly, path planning simulation experiments are conducted; the swarm algorithm performs global path pre-search, while the beetle algorithm handles local optimization. Two-dimensional, three-dimensional, and real-world environments are established to verify the engineering application value of the proposed algorithm.

This paper is structured into five parts. Section 1 provides a relevant introduction to the research content. Section 2 elaborates on the artificial bee colony algorithm, beetle swarm algorithm, and their respective improvement strategies. Section 3 focuses on testing the performance of the proposed algorithm. Section 4 presents path planning experiments conducted in 2D, 3D simulation environments, and a real-world environment. Section 5 includes conclusions and future prospects.

2. Hybrid Path Planning Based on Improved ABC and BAS Algorithms

2.1. ABC Algorithm for Global Path Exploration Optimization

2.1.1. The Optimized ABC Algorithm

The ABC algorithm model is optimized as an “employed–onlooker–scout” model, which makes full use of the contribution of each type of bee in the global nectar search process and determines the location of the next path point based on the fitness value. Given that the ABC algorithm excessively relies on the optimization capability of leading bees, it tends to bypass the exploration of fitness values in unknown regions by scout bees, which

leads to stagnation in the later stages of the search. To address this, the Tent chaotic perturbation mechanism and non-uniform mutation operator are introduced, respectively, aiming to enrich population diversity while enhancing the exploration capability of the search space, thereby generating more global waypoints available for selection by the algorithm.

Assuming there are A bees in a B -dimensional search space, the position of each bee is shown as

$$C = \begin{bmatrix} c_{1,1} & c_{1,2} & \dots & c_{1,b} & \dots & c_{1,B} \\ c_{2,1} & c_{2,2} & \dots & c_{2,b} & \dots & c_{2,B} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{d,1} & c_{d,2} & \dots & c_{d,b} & \dots & c_{d,B} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{A,1} & c_{A,2} & \dots & c_{A,b} & \dots & c_{A,B} \end{bmatrix}, \quad (1)$$

where $d = 1, 2, \dots, A$; $b = 1, 2, \dots, B$; and $c_{d,b}$ is the position of the d -th bee in dimension B .

The employed bees guide the movement of the entire honey colony and can search for nectar sources anywhere in the region. Their updated position is shown in

$$c_{d,b}^{e+1} = \begin{cases} c_{d,b}^e \exp\left(\frac{-b}{\alpha \cdot iter_{max}}\right), 0.5 \leq A \leq 1 \\ c_{d,b}^e + FG, 0 \leq A < 0.5 \end{cases}, \quad (2)$$

where e is the number of model iterations; $iter_{max}$ is the maximum number of iterations; α is a random number with $0 < \alpha \leq 1$; F is a random value obeying the normal distribution function; and G is a $1 \times b$ matrix where each element is 1. $A \in [0, 1]$, which denotes the concentration of nectar in the region. When $0 < A < 0.5$, the current concentration of nectar in the region is low, and the search range needs to be expanded. When $A \geq 0.5$, the current concentration of nectar is high, and all the honeybees need to fly to the region quickly.

The onlooker bees follow the employed bees in search of nectar and may harvest nectar together with the employed bees to enhance the nectar concentration of the entire population. Their updated position is shown in

$$c_{d,b}^{e+1} = \begin{cases} F \exp\left(\frac{cworst_b^e - c_{d,b}^e}{d^2}\right), d > \frac{A}{2} \\ cbest_b^{e+1} + \frac{1}{B} \sum_{b=1}^B \left(|c_{d,b}^e - cbest_b^{e+1}| \cdot rand\{-1, 1\}\right), d \leq \frac{A}{2} \end{cases}, \quad (3)$$

where $cworst_b^e$ is the global worst position of the bee in the b -th dimension at the e th iteration; $cbest_b^{e+1}$ is the global optimal position of the employed bee in the b -th dimension at the $(e + 1)$ -th iteration; and $rand$ is a randomly assigned value within the given range. When $d > A/2$, it means that the d th onlooker bee, which has low adaptability, is more likely to fail to collect honey due to the low nectar concentration in the region. Otherwise, the d -th onlooker bee finds a random position near the optimal nectar source where the employed bee is located to collect honey.

The random initial position of the scout bees is shown in

$$c_{d,b}^{e+1} = \begin{cases} cbest_b^e + \beta |c_{d,b}^e - cbest_b^e|, k_d \neq k_b \\ c_{d,b}^e + J \left(\frac{c_{d,b}^e - cworst_b^e}{|k_d - k_w| + \epsilon} \right), k_d = k_b \end{cases}, \quad (4)$$

where β denotes the step control parameter in the model, following a random normal distribution with a mean of 0 and variance of 1; J is a random value between $[-1, 1]$; k_d is

the current fitness value of the bees; k_b is the current optimal fitness value in the global search; k_w is the worst fitness value; and ε is a non-zero minimum constant. $k_d \neq k_b$ indicates that the scout bee is located at the edge of the population and $k_d = k_b$ indicates that the scout bee has found a better source of nectar and needs to move to a new region.

2.1.2. Improvement Strategies

The Tent chaotic function [26] is adopted to enrich the population diversity, prevent the ABC algorithm from falling into a premature convergence situation during the optimization process, and improve the algorithm's global search capability. The chaotic mapping function has strong traversal and randomness and can be mathematically mapped to transform chaotic sequences into individual spatial search capabilities. Among them, Tent chaos has a faster iteration speed and better uniformity, but there may be errors on immovable points. Therefore, random variables are introduced into the Tent chaos function to control the random values within a certain range and keep the algorithm's regularity. Firstly, during the initialization of the bee population, a random initial individual is generated as a b -dimensional vector, where each dimension is within the interval $[0,1]$. Subsequently, the next individual is generated by substituting the value of each dimension of the current individual into the Tent chaotic formula sequentially to update the corresponding dimension of the new individual. This process of generating new individuals is repeated until $(n - 1)$ new individuals are produced. All generated individuals are then mapped to the variable range, resulting in the Tent chaotic initial bee population. The initial population is generated as

$$z_{d+1} = \begin{cases} \frac{2z_d + \text{rand}(0,1)}{N}, & 0 \leq z_d \leq \frac{1}{2} \\ \frac{2(1-z_d) + \text{rand}(0,1)}{N}, & \frac{1}{2} < z_d \leq 1 \end{cases} \quad (5)$$

where N is the number of particles within the sequence, and $\text{rand}(0,1)$ takes a random number in that range.

A non-uniform mutation operator is introduced to enhance solution accuracy. The non-uniform mutation operator [27] can introduce large perturbations in the early iterations of the algorithm, thereby enhancing the exploration of the search space. In the later stages, it reduces the perturbation magnitude, promoting the algorithm's convergence to the vicinity of the global optimal solution and improving the quality of the global solution. For each bee generation after position update, the d -dimensional vector is randomly perturbed using a variance probability l_m , which decreases dynamically with the number of iterations. The mutation formulas for the o -th dimensional component are shown in

$$c_d^{o+1} = \begin{cases} c_d^o + \Delta(e, p_o - c_d^o), & \alpha = 0 \\ c_d^o - \Delta(e, c_d^o - q_o), & \alpha = 1 \end{cases} \quad (6)$$

$$\begin{cases} \Delta(o, r) = r \times (1 - s^{l_m^t}) \\ l_m = 1 - \frac{e}{E_{\max}} \end{cases} \quad (7)$$

where p_0 and q_0 are the maximum and minimum values of the vector range, respectively; α randomly takes the value 0 or 1; e is the current number of iterations; $E_{\max}$ is the maximum number of iterations; s is the number of random numbers in the interval $[0,1]$; and t determines the dependence degree of random number perturbation on the iteration count e . In the early stage of the algorithm, e is small, and the operator performs uniform search in the space. In the later stage, as e increases, the new solution generated by mutation infinitely approximates the optimal real solution, yielding the best nectar source.

2.2. Local Path Optimal Optimization Based on BAS Algorithm

2.2.1. The Optimized BAS Algorithm

The BAS algorithm is a novel heuristic algorithm based on simulating the mate and foraging behavior of beetles. If the ABC algorithm generates an excessive number of global waypoints, the BAS algorithm, with its strong local search capability, can obtain multiple locally optimal waypoints. This algorithm determines the optimal solution by comparing the fitness values of the current and the next waypoints. In this section, improvements are made to the optimization direction and fitness value confirmation of the BAS algorithm. The introduction of random reverse learning and a greedy selection mechanism enhances the reliability of the algorithm's waypoint selection and improves the accuracy of waypoint optimization. The beetles' search direction vector $\vec{d}$ is defined using the random search formula

$$\vec{d} = \left| \frac{\text{rand}(b, 1)}{\|\text{rand}(b, 1)\|} \right|, \quad (8)$$

where b is the spatial dimension.

The initial position of the beetle in the b -dimensional space is shown as follows:

$$c^0 = (c_1^0, c_2^0, \dots, c_b^0). \quad (9)$$

At the e -th iteration, the beetle's tentacles determine the left and right directions at the current position as follows:

$$\begin{cases} c_1^e = c^e + \frac{c_e^d}{2\vec{d}} \\ c_r^e = c^e - \frac{c_e^d}{2\vec{d}} \\ c_{e,d}^{d+1} = 0.95c_e^d + 0.01 \end{cases}, \quad (10)$$

where c_e^d denotes the distance between the two tentacles of the beetle at the e -th iteration; and $c_{e,d}^{d+1}$ is the updated length of the tentacles.

The beetle position of the next iteration is updated as

$$\begin{cases} c^{e+1} = c^e + \text{step}^e \vec{d} \text{sign}[k(c_1^e) - k(c_r^e)] \\ \text{step}^{e+1} = \mu \text{step}^e \end{cases}, \quad (11)$$

where step^e denotes the iterative step size of the beetle in the e -th iteration, which determines the convergence speed of the algorithm's optimization search; μ is the step factor; $\text{sign}(\cdot)$ is the sign function; and $k(c_1^e)$ and $k(c_r^e)$ denote the objective function values corresponding to the left and right directions at the e -th iteration, respectively. As the number of iterations increases, $k(c_r^e)$ continuously updates the convergence value until the optimal objective function solution is obtained.

2.2.2. The Improvement Strategy

The beetle algorithm, characterized by few parameters and a simple structure, tends to steer randomly in local paths. Therefore, positional stochastic inverse learning and a greedy selection strategy [28] are introduced to optimize the judgment of left and right directions. Currently, the beetle's direction selection is limited to left and right, with the remaining position information being underutilized. A local search in nearby directions is required to find a better position and avoid random steering. The stochastic inverse learning method generates a stochastic inverse solution based on the left and right directions, thereby increasing direction options and avoiding blind movement, as shown in

$$c_{\text{new}}^e = c_1^e + c_r^e - c_{\text{rand}} \times c_{\text{best}}^e, \quad (12)$$

where c_{new}^e denotes the newly generated random direction and c_{best}^e is the current optimal position.

Since the fitness value of the generated new position may be lower than those of the left and right directions, a greedy selection of the beetle's direction is performed to expand the search range and avoid local optima. In each iteration, the fitness values of the generated new position direction and the original left and right directions are compared. If the fitness value of the new direction is better than those of the original left and right directions, the beetle moves toward the new direction for updating as per Equation (12); otherwise, it moves as Equation (11). The next optimal position of the beetle is obtained through iterative screening using this method, as shown in

$$kc^{e+1} = \begin{cases} k(c_{new}^e), k(c_{new}^e) > k(c_1^e) - k(c_r^e) \\ k(c_1^e), k(c_1^e) - k(c_r^e) > k(c_{best}^e) \\ k(c_r^e), k(c_1^e) - k(c_r^e) < k(c_{best}^e) \end{cases} \quad (13)$$

2.3. Adaptive Adjustment to Algorithm

In the early stage of the improved algorithm, the number of individual bees in the population is large and scattered. Adaptively adjusting the ABC algorithm to obtain a larger weight value can accelerate the convergence process and enhance the ability to explore global waypoints. As the number of iterations increases, the weight of the ABC algorithm gradually decreases, while the weight of the BAS algorithm gradually increases. This helps to refine the search for local waypoints and find the optimal path. The nonlinear inertia weight factor [29] is introduced to adaptively adjust the weights of the two algorithms and achieve a balance between the global exploration and local optimality search. The nonlinear weight factor update formula is shown in

$$\omega = \omega_{\min} + (\omega_{\max} - \omega_{\min}) \cdot \sin\left(\frac{\pi}{2} \cdot \frac{e}{e_{\max}}\right). \quad (14)$$

The values of $\omega_{\max} = 0.84$ and $\omega_{\min} = 0.01$ are determined through extensive experiments. The traditional adaptive weight factor increases linearly, whereas the nonlinear weight factor employed here rises slowly in the early iterations, expanding the search range and thereby enhancing global exploration capability. In the later iterations, the increase rate of the weight factor accelerates, further strengthening the local exploitation capability. The variation in ω is illustrated in Figure 1.

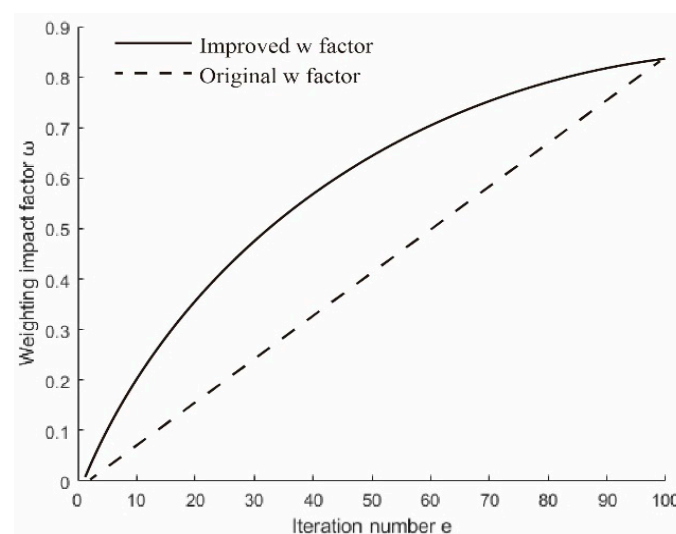


Figure 1. Comparison of adaptive weight factors.

An adaptive nonlinear inertia weight is incorporated into the position update mechanisms of both the swarm and beetle algorithms, as shown in

$$c_{d,b}^{e+1} = \begin{cases} w \times c_{d,b}^e \exp\left(\frac{-b}{\alpha \cdot dter_{\max}}\right) \\ cbest_b^{e+1} + \frac{1}{B} \left(\sum_{b=1}^B |w \times c_{d,b}^e - cbest_b^{e+1}| \cdot \text{rand}\{-1, 1\} \right) \\ c_{d,b}^e + J\left(\frac{w \times c_{d,b}^e - cworse_b^e}{|k_d - k_w| + \varepsilon}\right) \end{cases}, \quad (15)$$

$$c_b^{e+1} = w \times c^e + \text{step}^e \vec{d} \text{sign}[k(c_1^e) - k(c_r^e)]. \quad (16)$$

A dynamic coefficient ε is introduced to adjust the proportion of the two algorithms, regulate the ratio between current weights and weight changes from the previous generation during each weight update, and achieve a dynamic balance between global exploration and local exploitation through iterations, as shown in

$$\begin{cases} c^{e+1} = (1 - \varepsilon)c_{d,b}^{e+1} + \varepsilon c_b^{e+1} \\ \varepsilon = \frac{e}{2E} \end{cases}. \quad (17)$$

2.4. Path Planning Based on IABCBAS Algorithm

The path planning process based on the IABCBAS algorithm is illustrated in Figure 2, with the steps as follows:

- (1) Set the relevant parameters for the ABC algorithm and generate the bee population with Tent chaotic perturbation using Equation (5).
- (2) “Leading” and “following” are set as the main behaviors, interspersed with scouting, to form a new distribution of the honeybee population; obtain the corresponding nectar concentration, and select the colony distribution position based on nectar concentration.
- (3) Mutate the bees after position update using Equations (6) and (7). If the nectar concentration after mutation is higher than that before mutation, update the position to the mutated one; otherwise, no operation is performed.
- (4) If the maximum number of iterations of the bee colony algorithm is not reached, repeat the above cycle; otherwise, increase the initial pheromone value of the path points corresponding to the current bee colony to form an optimized initial pheromone distribution and improve the quality of global search points.
- (5) Set the relevant parameters of the BAS algorithm and perform local numerical optimization using the optimized initial pheromone.
- (6) Randomly generate a new solution using Equation (12) and calculate the fitness value of it. If the new solution outperforms the previous one, the direction of the optimization search is adjusted accordingly.
- (7) The optimal solution is retained according to Equation (13), with local and global pheromones updated simultaneously.
- (8) Check if the maximum number of iterations of the BAS algorithm has been reached. If so, the results of the optimal search are saved, analyzed, and processed.

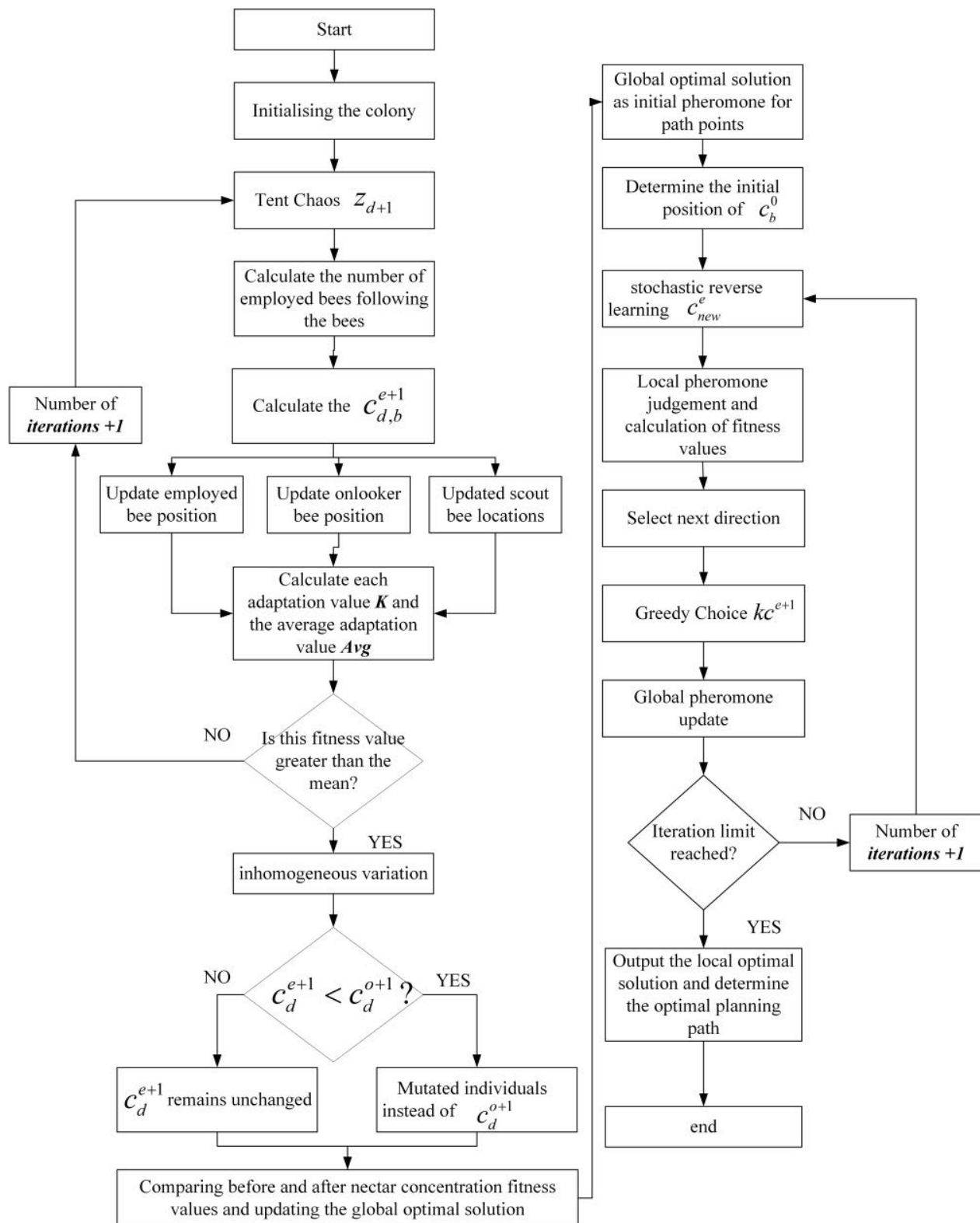


Figure 2. The flow path of the IABCBAS algorithm.

2.5. Analysis of Algorithm Time Complexity

The time complexity of an algorithm can reflect its feasibility and efficiency in solving path planning problems. Therefore, this section conducts a time complexity analysis of the original ABC and BAS algorithms, as well as the improved IABCBAS algorithm.

In the global waypoint exploration phase, the size of the artificial bee colony is C , the problem dimension is b , and the maximum number of iterations of the algorithm

is E . Therefore, the time complexity of the original ABC algorithm is $O(c \times b \times E)$. The initialized population of the improved ABC algorithm is disturbed by Tent chaos, and the initialization time complexity is $O(b)$. Affected by the mutation factor, the time complexity of the waypoint exploration phase is $O(c \times b \times E)$. Thus, the time complexity of the improved ABC algorithm phase is $O(b + c \times b \times E)$.

In the local waypoint optimization phase, the problem dimension at this stage is b . The computational complexity of initializing the beetle is $O(b)$. During the iteration process, the computational complexity for judging the directions of the two antennae is $O(2b)$, and the computational complexity for updating the beetle's position is $O(2b)$. Therefore, the time complexity of the original BAS algorithm is $O(5b)$. Since the initial position of the beetle in the improved BAS algorithm is generated in the global exploration phase, the initialization computational complexity is reduced. Due to the introduction of random reverse solutions, the computational complexity caused by the increased direction selection is $O(3b)$. The computational complexity of greedy selection confirmation is $O(b)$. Thus, the time complexity of the improved BAS algorithm is $O(4b)$.

In summary, the combined time complexity of the two original algorithms before improvement is $O(c \times b \times E) + O(5b) = O(c \times b \times E + 5b)$, and the complexity of the IABCBAS algorithm is $O(b + c \times b \times E) + O(4b) = O(c \times b \times E + 5b)$. It can be seen that the IABCBAS algorithm has the same complexity of $O(c \times b \times E + 5b)$. However, through the design of optimization strategies, the IABCBAS algorithm improves the search capability, controls the time, and improves the efficiency of path planning.

3. Experimental Testing of Search Performance on IABCBAS Algorithms

3.1. Test Functions and Environment

Four swarm intelligence algorithms from different periods are selected to conduct comparative experiments on nine test benchmark functions to verify the performance and feasibility of the proposed IABCBAS algorithms. All experiments are conducted on the same experimental platform. The comparison algorithms [16] and related parameters are shown in Table 1.

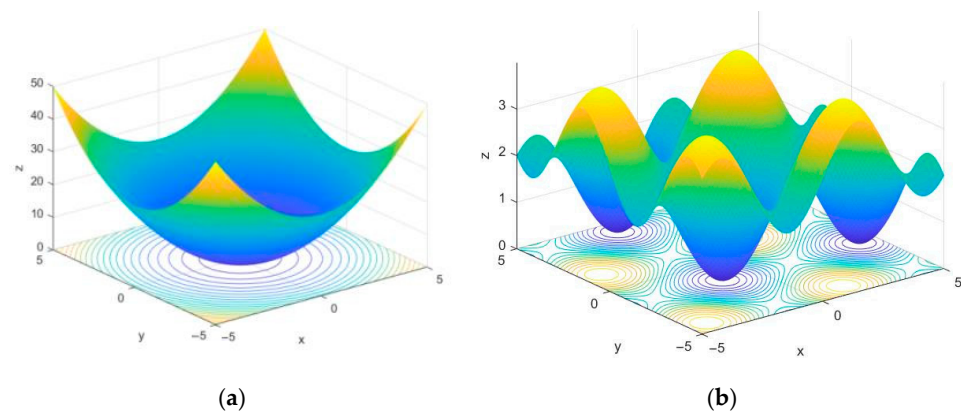
Table 1. The comparison algorithms and related parameters.

Algorithm	Parameter Setting	Parameter Meaning
PSO	$c_1 = c_2 = 2$ $\omega \in [0.5, 0.8]$	c_1, c_2 are learning factor, ω is inertia weight, adjustment of particle speed
ACO	$\alpha = 1 \quad \beta = 3 \quad p = 0.6$	α is pheromone heuristic factor, β is heuristic function factor, p is pheromone evaporation coefficient
SSA	$P_{dp} = 0.2 \quad G_c = 1.8 \quad sf = 18$	P_{dp} is sparrows exploring new areas, G_c is sparrows that warn of danger, sf is alert threshold
WOA	$\alpha_s = 0.15 \quad \varepsilon = 0.01 \quad b = 2$	α_s is linear control parameter, ε is hunting probability factor, b is bubble spiral sparsity

Benchmark functions with different dimensions and numbers of peaks are employed to verify the algorithms' convergence speed and their ability to escape local optima. The selected benchmark functions [30] are listed in Table 2, and part of the test function space is illustrated in Figure 3. The Sphere test function is a continuous unimodal convex function in a high-dimensional space with multiple local minima, enabling it to test the search algorithm's ability to escape local minima. The Foxholes test function is a low-dimensional multimodal function with numerous canyons and obstacles, making the testing of algorithms more challenging.

Table 2. The nine benchmark parameters for algorithm testing.

Type	Parameter	Basis Function	Function	Dimension	Corridor	Optimum Value
Higher dimensional unimodal function	f_1	Schwefel's	$f_1(x) = \sum_{i=1}^N x_i + \prod_{i=1}^N x_i $	30	$[-10,10]$	0
	f_2	Rosenbrock	$f_2(x) = \sum_{i=1}^{N-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$	30	$[-30,30]$	0
	f_3	Sphere	$f_3(x) = \sum_{i=1}^N x_i^2$	30	$[-100,100]$	0
Higher dimensional multimodal function	f_4	Rastrigrin	$f_4(x) = \sum_{i=1}^{N-1} [x_i^2 - 10\cos(2\pi x_i) + 10]$	30	$[-5.12,5.12]$	0
	f_5	Schwefel	$f_5(x) = \sum_{i=1}^N -x_i \sin(\sqrt{ x_i })$	30	$[-500,500]$	−418.98
	f_6	Griewing	$f_6(x) = \frac{1}{4000} \sum_{i=1}^N x_i^2 - \prod_{i=1}^N \cos(\frac{x_i}{\sqrt{i}}) + 1$	30	$[-600,600]$	0
Low-dimensional multi-maximilian function	f_7	Foxholes	$f_7(x) = \left(\frac{1}{500} + \sum_{j=1}^{25} \frac{1}{j + \sum_{i=1}^2 (x_i - a_{ij})^6} \right)^{-1}$	2	$[-65,65]$	1
	f_8	Schkel	$f_8(x) = -\sum_{i=1}^{10} \left[(x - a_i)(x - a_i)^T + c_i \right]^{-1}$	4	$[-10.54,10.54]$	−10.54
	f_9	Hartmann 6-D	$f_9(x) = -\sum_{i=1}^4 c_i \exp\left(-\sum_j a_{ij} (x_j - p_{ij})^2\right)$	6	$[-3.52,0]$	−3.32


Figure 3. Benchmark test function modeling. (a) Sphere function; (b) Foxholes function.

3.2. Convergence and Stability Analysis

Performance comparison experiments are conducted on five algorithms and each benchmark function is run independently 50 times. The evaluation metrics include the best value, average value, and standard deviation. The population size is set to 40 and the maximum number of iterations is 500. The experimental results are presented in Table 3.

Among these functions, high-dimensional unimodal functions have only one global optimal point and no local extrema, which are mainly used to test convergence speed. Multimodal functions, which have multiple local extrema, are used to evaluate the performance of escaping local extrema across different dimensions (both high and low). For the three high-dimensional unimodal functions, the proposed IABCBAS algorithm achieves the theoretical optimal value of 0 when solving f_1 and f_3 , with the corresponding standard deviation also being 0. In solving f_4 , the optimal value, average value, and standard deviation of the proposed IABCBAS algorithm are at least three orders of magnitude better than those of the other four algorithms.

Table 3. Comparison of benchmark function test results.

Norm	Algorithms	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9
Optimal value	PSO	5.684×10^{-4}	1.635×10^{-1}	1.527×10^{-7}	2.906	-8.207×10^3	2.453×10^{-7}	0.8909	−10.54	−3.322
	ACO	3.291×10^{-15}	2.661×10^{-1}	3.206×10^{-24}	0	-8.756×10^3	0	1	−10.54	−3.322
	WOA	4.672×10^{-51}	2.808×10^{-1}	3.579×10^{-49}	0	-1.972×10^4	0	0.8909	−10.54	−3.322
	SSA	1.738×10^{-97}	6.431×10^{-8}	0	0	-9.816×10^3	0	1	−10.54	−3.322
	Our	0	1.856×10^{-11}	0	0	-5.839×10^3	0	1	−10.54	−3.322
Average value	PSO	4.915×10^{-3}	6.159×10^{-1}	1.822×10^{-6}	5.206	-5.106×10^3	1.731×10^{-2}	4.536	−9.561	−3.472
	ACO	7.693×10^{-15}	2.993×10^{-1}	2.255×10^{-21}	3.714	-6.658×10^3	3.545×10^{-3}	3.729	−10.33	−3.452
	WOA	4.798×10^{-42}	3.097×10^{-1}	5.304×10^{-34}	0	-1.507×10^4	0	4.883	−7.911	−3.409
	SSA	1.659×10^{-11}	4.391×10^{-6}	2.727×10^{-61}	0	-8.584×10^3	0	3.487	−8.805	−3.482
	Our	3.148×10^{-26}	2.178×10^{-9}	0	0	-4.541×10^3	0	3.119	−10.006	−3.421
Standard deviation	PSO	6.161×10^{-3}	5.179×10^{-1}	2.301×10^{-6}	1.964	1.801×10^3	9.233×10^{-4}	5.196	2.508	5.902×10^{-2}
	ACO	4.908×10^{-15}	5.476×10^{-1}	2.899×10^{-21}	4.241	9.407×10^2	8.105×10^{-4}	4.184	1.382	8.541×10^{-2}
	WOA	7.056×10^{-42}	4.426×10^{-1}	4.375×10^{-33}	0	1.679×10^3	0	5.125	3.683	1.185×10^{-1}
	SSA	3.851×10^{-12}	2.414×10^{-5}	3.163×10^{-61}	0	5.154×10^2	0	3.422	2.256	5.741×10^{-2}
	Our	0	3.707×10^{-8}	0	0	4.724×10^3	0	2.692	3.086×10^{-11}	3.411×10^{-2}
Running time	PSO	4.691	4.006	3.821	4.225	3.712	3.731	4.115	3.301	4.347
	ACO	5.155	4.802	4.963	4.821	4.267	4.446	4.297	3.633	4.679
	WOA	8.403	6.308	5.889	5.563	5.953	5.274	4.708	4.037	4.239
	SSA	5.323	4.702	4.521	4.227	4.832	4.389	3.813	3.691	4.621
	Our	4.469	4.128	3.689	4.043	3.406	3.385	3.729	3.415	4.386

Among the three high-dimensional multimodal functions, the WOA and SSA find the optimal values for f_4 and f_6 , and the proposed IABCBAS algorithm maintains superior test performance; the proposed algorithm has much higher values for the f_5 function than that of the other four algorithms, reflecting improved search capability. For the three low-dimensional multimodal functions f_7 to f_9 , our algorithm reaches the optimal value under the solution, demonstrating better optimization performance under low-dimensional conditions. In terms of stability, only the ACO, SSA, and our algorithm find the optimal solution for f_7 , with their stability increasing in sequence. Our algorithm has the best stability for f_8 and ranks second only to the WOA for f_9 . Across all algorithms, the accuracy on unimodal functions is higher than that on multimodal functions, and the accuracy on low-dimensional functions is higher than that on high-dimensional functions. The proposed IABCBAS algorithm achieves superior performance in both accuracy and stability. Regarding running time, different search mechanisms result in significant differences among algorithms. The average running times of the five algorithms are 3.994, 4.563, 5.597, 4.458, and 3.889 s, respectively. The proposed IABCBAS algorithm can generate multiple distinct solutions simultaneously, accelerating numerical comparisons and improving running time by 2.7% compared to the fastest PSO algorithm.

The average convergence curves of the nine benchmark test functions are plotted to compare the convergence speed, accuracy, and ability to escape local optima among different algorithms, as shown in Figure 4. The vertical axis represents the fitness value, and the horizontal axis denotes the iteration number. Figure 4a–c demonstrate that the proposed IABCBAS algorithm exhibits faster convergence speed in unimodal functions, attributed to the chaotic initialization of the population. Simultaneously, the convergence accuracy of this algorithm significantly outperforms others. In both (a) and (c), the proposed algorithm escapes local extrema, whereas PSO, ACO, and WOA fail to do so. In the high-dimensional multimodal functions, ACO, WOA, SSA, and our algorithm achieve the optimal value of 0 in f_4 and f_6 ; thus, the convergence curves are not given due to early convergence. It is worth noting that our algorithm requires significantly fewer iterations than the other three algorithms and exhibits superior convergence performance. Figure 4d shows that the proposed IABCBAS algorithm falls into the local extreme value point in the 200th–300th generation, and there is a significant difference between the optimization value and the optimal value after 300 generations. However, the overall numerical curve amplitude is smaller and more stable than the other four algorithms. In the low-dimensional multimodal function, the numerical types of functions f_8 and f_9 are the same; only the former is shown in Figure 4f. All algorithms in Figure 4e,f converge to the optimal solution, but the proposed IABCBAS algorithm exhibits the fastest convergence speed.

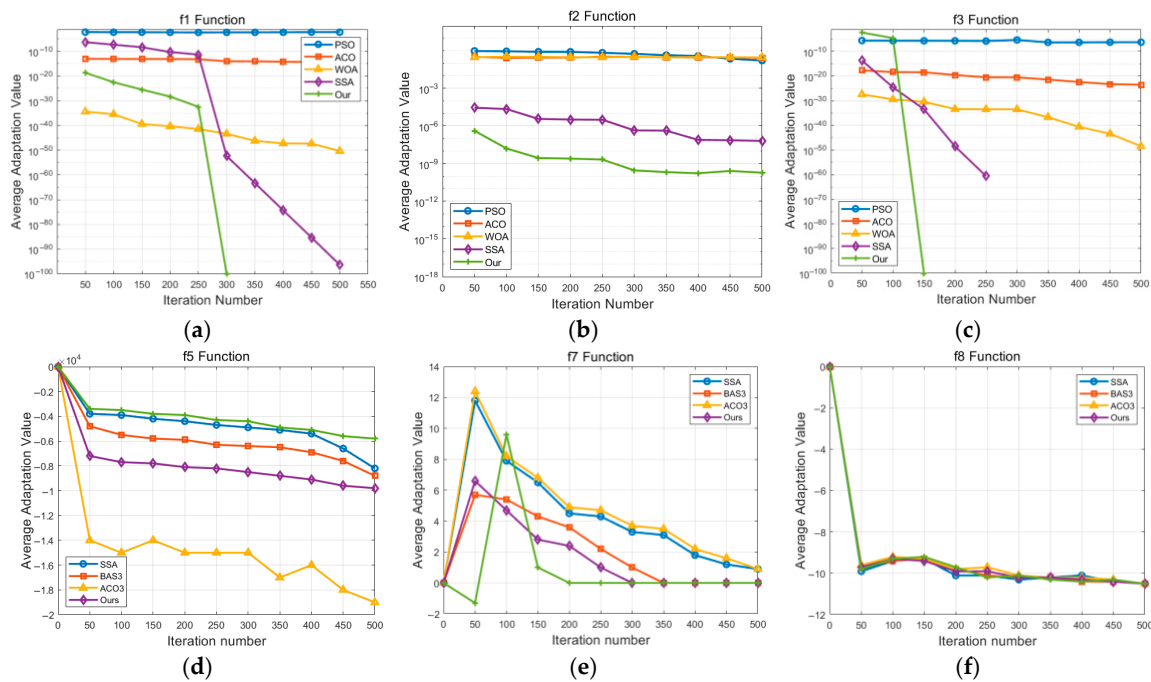


Figure 4. Comparison of benchmark function convergence curves. (a) Schwefel's function; (b) Rosenbrock function; (c) Sphere function; (d) Schwefel function; (e) Foxholes function; (f) Schkel function.

3.3. Comparative Ablation Experiments

The proposed algorithm has been proven to exhibit superior global search capability and local optimization performance. However, additional experiments are required to verify (1) whether the selected mechanism is optimal, (2) whether it represents the optimal improvement for the proposed algorithm, and (3) whether the combination of the two algorithms is generalizable. To enhance the persuasiveness of model improvements, comparative algorithms are selected for experiments by reviewing the latest literature. The ABC algorithm is modified by introducing Bernoulli chaotic mapping, reverse learning mechanism, and KHO algorithm, respectively [29–31], denoted as ACO1–ACO3; the BAS algorithm is selected to combine with ABC, PSO, and AFSA [32–34], denoted as BAS1–BAS3. ACO1 and ACO2 were set up to compare with the Tent population Chaos and local random direction selection mechanism. ACO3 and BAS1–BAS3 were set up to compare the two algorithm-combining methods we used, respectively. The results are shown in Table 4.

It can be seen from Table 3 that compared with the original ABC algorithm, the added mechanisms have improved the algorithm's optimality. The numerical results of ACO1 are all better than those of ACO2 in $f_1 \sim f_6$ function, indicating that enhancing population richness can effectively improve global search efficiency in high-dimensional functions. The numerical results of ACO2 are slightly better than that of ACO1 in function $f_7 \sim f_9$, indicating that increasing the random direction can increase local search stability in the low-dimensional function. Comprehensive results show that the results of each two combined algorithms are numerically better than the single-added mechanism results. The numerical results of BAS3 and ACO3 are better than those of BAS1 and BAS2, indicating that the combination of the new swarm intelligence algorithm has a wider search range and better enhancement effect. Meanwhile, the overall value of BAS1 is better than BAS2, indicating that the combination of the bee swarm and the beetle swarm is also good, which side by side verifies the accuracy of the basic algorithm selected. However, compared to the fish swarm and krill swarm algorithms, the beetle swarm algorithm has a simpler structure, giving the IABCBAS algorithm a greater advantage in search and optimization capabilities.

Furthermore, taking the f_1 function as an example, the average value of the IABCBA algorithm is the lowest among the eight algorithms, reaching 4.076×10^{-29} , which indicates that the algorithm has a better optimization effect on the f_1 function. Meanwhile, only the IABCBA algorithm has a standard deviation of 0, suggesting that the results of multiple runs are highly stable without significant differences.

Table 4. Test results of each algorithm on the benchmark function.

Algorithm	f_1 Function		f_2 Function		f_3 Function	
	Mean	Std	Mean	Std	Mean	Std
ABC	3.243×10^{-9}	5.003×10^{-8}	3.132×10^{-1}	4.233×10^{-1}	4.431×10^{-4}	3.165×10^{-3}
ACO1	1.605×10^{-20}	2.327×10^{-19}	4.292×10^{-2}	3.482×10^{-2}	2.076×10^{-8}	1.297×10^{-7}
ACO2	1.763×10^{-17}	2.301×10^{-16}	3.914×10^{-3}	2.631×10^{-2}	2.435×10^{-8}	1.371×10^{-7}
ACO3	3.363×10^{-27}	2.092×10^{-26}	2.472×10^{-11}	4.972×10^{-10}	6.316×10^{-31}	2.124×10^{-30}
BAS1	1.026×10^{-22}	1.924×10^{-21}	5.594×10^{-7}	2.741×10^{-6}	4.291×10^{-19}	1.137×10^{-18}
BAS2	5.084×10^{-21}	5.221×10^{-20}	6.357×10^{-6}	4.738×10^{-5}	4.578×10^{-18}	1.119×10^{-17}
BAS3	5.675×10^{-26}	3.929×10^{-25}	4.771×10^{-10}	3.829×10^{-9}	5.581×10^{-24}	2.186×10^{-23}
IABCBA	4.076×10^{-29}	0	3.122×10^{-13}	2.891×10^{-12}	0	0

Algorithm	f_4 function		f_5 function		f_6 function	
	Mean	Std	Mean	Std	Mean	Std
ABC	2.698	1.627	-3.133×10^2	3.156×10^2	1.957×10^{-2}	6.452×10^{-4}
ACO1	2.063	1.614	-4.497×10^2	3.661×10^2	1.204×10^{-2}	7.442×10^{-4}
ACO2	2.274	2.014	-4.398×10^2	3.349×10^2	1.705×10^{-2}	7.452×10^{-4}
ACO3	0	0	-8.367×10^2	8.427×10^2	0	0
BAS1	0	0	-6.823×10^2	5.391×10^2	0	0
BAS2	0	0	-6.332×10^2	5.665×10^2	0	0
BAS3	0	0	-7.422×10^2	7.308×10^2	0	0
IABCBA	0	0	-5.282×10^3	5.511×10^3	0	0

Algorithm	f_7 function		f_8 function		f_9 function	
	Mean	Std	Mean	Std	Mean	Std
ABC	5.317	4.223	−7.868	2.718	−3.224	0.0596
ACO1	5.479	4.604	−6.312	3.117	−3.168	0.0617
ACO2	5.328	4.535	−6.653	3.129	−3.172	0.0612
ACO3	4.263	3.557	−9.938	1.219	−3.396	0.0432
BAS1	4.812	4.134	−8.362	2.346	−3.254	0.0569
BAS2	4.766	4.117	−8.504	2.512	−3.298	0.0584
BAS3	4.339	3.692	−9.556	1.532	−3.387	0.0479
IABCBA	3.218	2.692	−10.006	0.825	−3.407	0.0363

4. Simulation Experiment of IABCBA Algorithm Path Planning Application

4.1. Establishment of Environmental Models

The IABCBA algorithm is capable of rapidly generating a large number of waypoints, conducting optimal path searches, and enabling mobile platforms to avoid obstacles in the environment through path planning, thereby enhancing the operational efficiency and environmental adaptability of the platforms. Particularly in path planning for field environments such as military operations, this algorithm can give full play to the advantages of search algorithms in solving high-dimensional space problems; it efficiently explores the optimal solutions for path planning in three-dimensional space and optimizes the kinematic constraints and performance constraints of mobile platforms. Therefore, the IABCBA algorithm holds high application value in addressing path planning problems across mul-

multiple engineering application fields. Two-dimensional fixed-area and three-dimensional simulation map models are established based on different obstacle distributions in various environments to verify the universality of the proposed method, as shown in Figure 5. For the 2D environment, a simple map (20×20) and a complex map (30×30) are constructed, where obstacles are represented by black grids and free spaces by white grids. For the 3D environment, simple bimodal and complex multimodal elevation maps are built using elevation data.

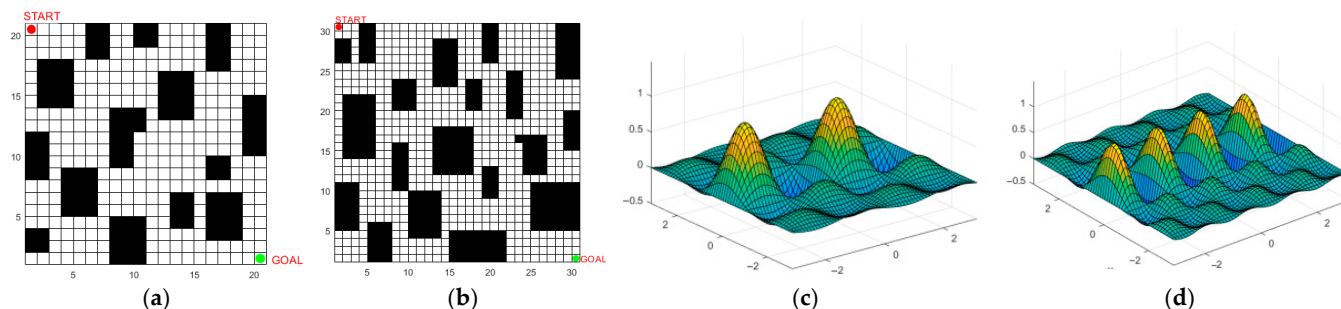


Figure 5. Establishment of environmental map model. (a) 20×20 ; (b) 30×30 ; (c) twin peaks terrain; (d) multimodal terrain.

In Figure 5a,b, the top left corner of the map is the starting point of the path planning and the bottom right corner is the endpoint. The fence side length is set as B , the fence space is divided into M_x rows and M_y columns with B unit length. The n -th fence grid coordinates are (x_n, y_n) , and the formula is shown in

$$\begin{cases} x_n = B \times ((i-1) \bmod M_x) + \frac{B}{2} \\ y_n = B \times \left(M_y - \text{ceil}\left(\frac{i}{M_y}\right) \right) + \frac{B}{2} \end{cases} \quad (18)$$

where $\bmod$ is the redundancy function and ceil is the upward rounding function.

The real coordinates are scaled down to one-tenth of their original values for the axes in Figure 5c,d to facilitate the 3D environment modeling. The slope model is generated based on the average slope between adjacent path points, as shown in Equation (19) and Figure 6.

$$\theta_{1xy} = \arccos \frac{b_y - b_x}{d_{xy}}, \quad (19)$$

where x and y are the coordinates corresponding to adjacent elevation path points in the B -dimensional space, θ_{1xy} is the average slope of the path point b_y relative to path point b_x , and d_{xy} is the planar straight-line distance between the two path points.

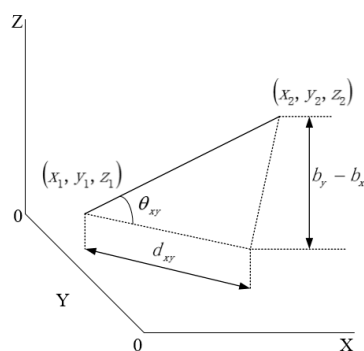


Figure 6. Schematic diagram of slope model.

4.2. Path Modeling

The path obtained by the search algorithm from the start point to the end point in the 2D grid map can be regarded as a set T of path points during motion planning. Path planning requires the vehicle to travel from the start point to the goal point, avoiding all obstacles and finding the shortest path, as shown in

$$T = \{S, U_1, U_2, \dots, U_{M-1}, G\}, \quad (20)$$

$$\text{Length}T = \|U_1 - S\| + \sum_{i=1}^{M-1} \|U_{i+1} - U_i\| + \|U_{M-1} - G\|, \quad (21)$$

$$\begin{cases} T_{\min} = \text{argmin}(\text{Length}_v) \\ P(\text{Length}_v) = 0 \end{cases}, \quad (22)$$

where S is the start point (x_0, y_0) ; $U_1, U_2, \dots, U_{M-1}$ is the via path node; G is the goal point (x_M, y_M) ; $T_{\min}$ is the minimum value of the search path; Length_v is the shortest planned path v ; and $P(\text{Length}_v)$ is the probability of collision between the path and the obstacle.

The path points are also treated as a series of discrete points in the 3D environment. The next movable path point is generated once the movement direction DIR is determined, as shown in Equation (23). The optimal path point that avoids obstacles is identified, as shown in Equation (24), and added to the shortest path $T_{\min} \text{Length}$.

$$\begin{cases} x_{i+1} = x_i + \text{dir}_x \times \text{rand}q_{1xy} \\ y_{i+1} = y_i + \text{dir}_y \times \text{rand}q_{1xy} \\ z_{i+1} = z_i + \text{dir}_z \times \text{rand}q_{1xy} \end{cases}, \quad (23)$$

$$\begin{cases} U_{\min} = \sqrt{(U_1 + U_2) \times \text{pri}} \\ U_1 = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2 + (z_{i+1} - z_i)^2} \\ U_2 = \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2 + (z_i - z_{i-1})^2} \\ T_{\min} \text{Length} = \sum_{i=0}^B u_i \end{cases}, \quad (24)$$

where U_1 and U_2 are the Euclidean distances between the current and the next path point and endpoint, respectively; and Pri is analogous to $P(\text{Length}_v)$, which avoids the searched path points colliding with obstacles in the map.

4.3. Comparison Experiment of Two-Dimensional Environment Simulation

In the initial stage, the improved ABC algorithm performs global search to generate multiple feasible paths, thereby enhancing the global search capability and verifying the effectiveness of path planning. The fitness value is calculated to determine the highest-quality global path, and the path points corresponding to the pheromone distribution are passed to the BAS algorithm. This reduces the blindness of the BAS algorithm's search, strengthens its local optimization capability, and accelerates convergence.

The SSA, ACO3, and BAS3 algorithms are selected for comparative simulation experiments on path planning. Each algorithm runs 50 experiments in the fence map in Figure 5, with the following parameters: maximum iteration number = 500; search space dimension = 30; and population size = 100. All experiments are conducted under identical parameters. The path planning results in simple and complex environments are compared in Figures 7 and 8, while the convergence comparison of different algorithms in these two environments is presented in Figure 9.

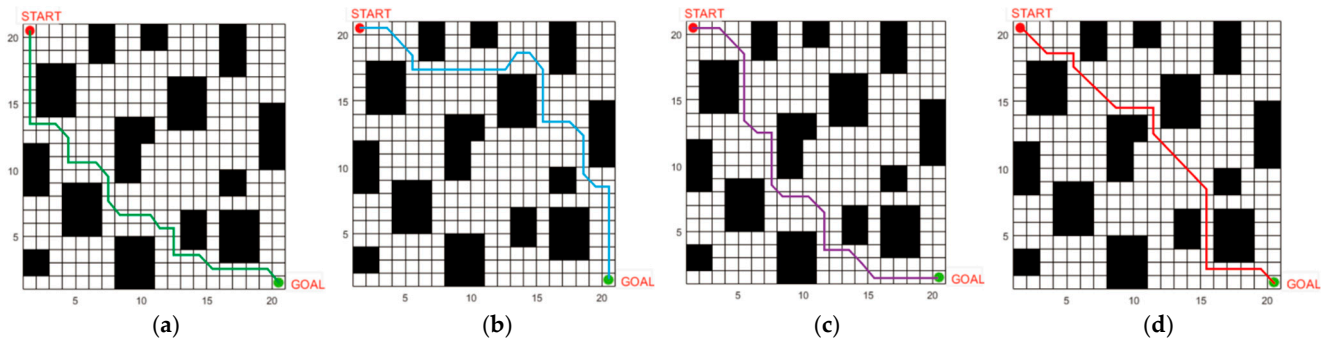


Figure 7. Optimal path planning results of four algorithms in a 20×20 2D simple environment: (a) SSA; (b) BAS3; (c) ACO3; (d) IACBAS.

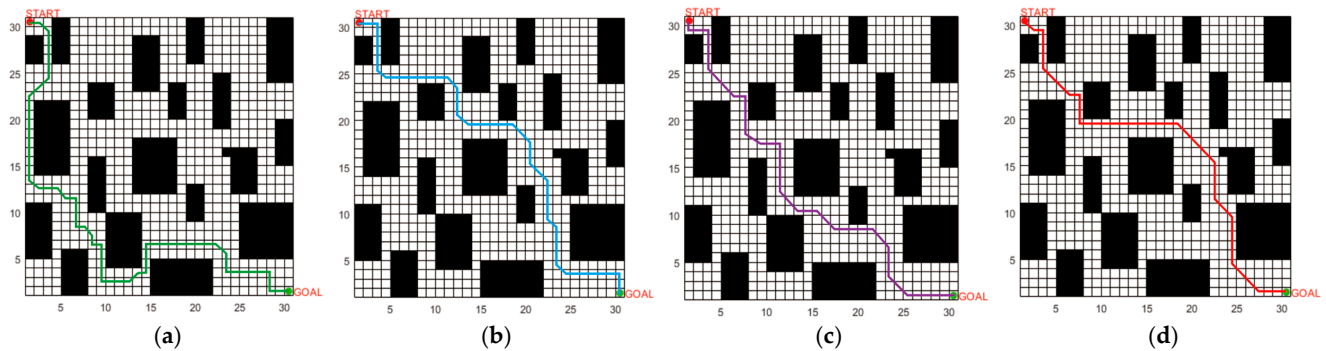


Figure 8. Optimal path planning results of four algorithms in a 30×30 2D complex environment: (a) SSA; (b) BAS3; (c) ACO3; (d) IACBAS.

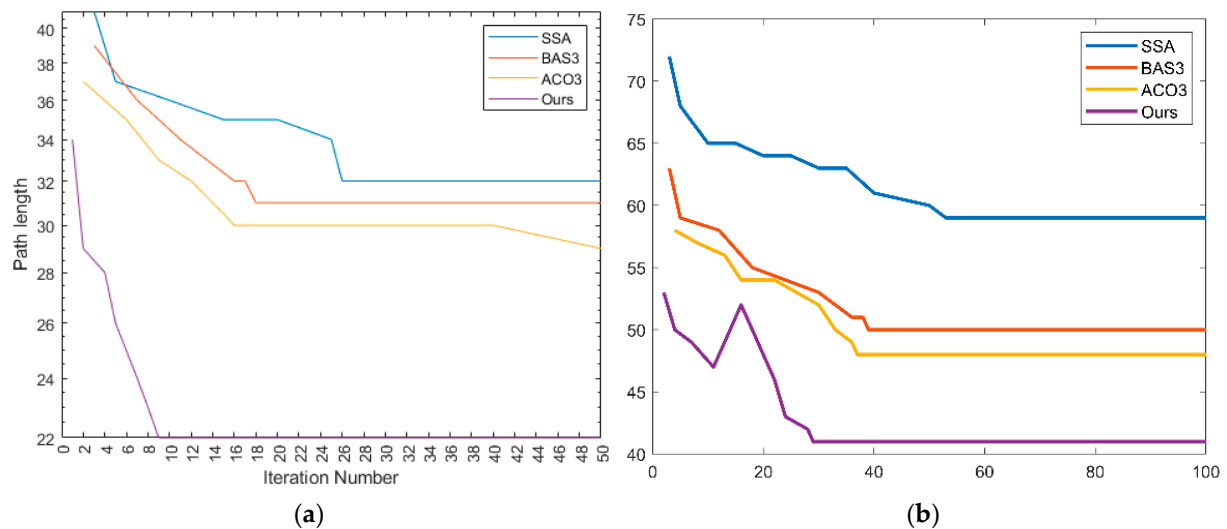


Figure 9. Convergence of four algorithmic paths in 2D environments: (a) 20×20 environmental convergence effect; (b) 30×30 environmental convergence effect.

As observed in Figures 7 and 9, for path planning in the 20×20 simple environment map, the SSA converges to the optimal solution at the 26th iteration, with an optimal path length of 32.4. However, the path contains numerous turns, which is unfavorable for the operation of unmanned vehicles in real environments. The BAS3 and ACO3 algorithms converge to the optimal solution successively at around the 18th and 17th iterations, with optimal path lengths of 29.8 and 26.4, respectively. Compared with the SSA, both exhibit fewer turns; however, their paths turn near obstacles, indicating weak space utilization capability. The proposed algorithm converges to the optimal solution at the ninth iteration,

with a shorter planned path that is not affected by obstacles in space, thus achieving the best overall planning performance.

The comparison of the average values of the four algorithms over 500 iterations is presented in Table 5. For path planning in the complex environment map, the convergence times of the SSA, BAS3, and ACO3 algorithms increase with the number of obstacles, converging to the optimal solution at the 49th, 42nd, and 39th iterations, respectively. Additionally, the number of turns increases, and the path length becomes longer. The proposed IABCBAS algorithm yields an overall smoother path. Compared with SSA, BAS3, and ACO3, the number of turns is reduced by 46.32%, 28.74%, and 25.31%, respectively. Moreover, the planning time for the shortest path is 10.28 s faster than the average of the three algorithms. This indicates that the algorithm maintains superior planning performance in complex environments, meeting the current requirements of engineering applications.

Table 5. Comparison of average performance of four algorithms in two-dimensional environments.

Environments	Method	Average Path Length (m)	Average Number of Turns	Average Number of Iterations	Average Planning Time (s)
20 × 20 2D simple environment	SSA	34.8	16.3	28.5	15.58
	BAS3	31.4	13.2	21.2	11.59
	ACO3	29.7	12.5	19.7	10.77
	IABCBAS	26.2	9.8	11.1	6.07
30 × 30 2D complex environment	SSA	62.6	23.1	49.3	39.28
	BAS3	52.9	17.4	41.8	33.31
	ACO3	50.5	16.6	39.4	31.39
	IABCBAS	44.3	12.4	30.6	24.38

4.4. Comparison Experiment of Three-Dimensional Environment Simulation

In this section, 3D path planning experiments are conducted on maps constructed from 3D elevation data to derive more comprehensive empirical conclusions and verify the adaptability of the proposed algorithm to real complex environments in 3D path planning. The comparison algorithms remain SSA, ACO3, and BAS3. For experimental data processing, the values in the simulation coordinates are multiplied by ten to convert them to real environment values. The experiment is repeated 30 times, and the average value of each index calculated by each algorithm is recorded to eliminate the interference of random errors. The experimental results are shown in Table 5.

The results of optimal 3D path planning among different algorithms in the two environments are illustrated in Figure 10. The convergence comparison of different algorithms in these environments is presented in Figure 11. As can be seen from Figures 10 and 11, when path planning in a 3D unimodal simple environment map, the SSA converges to the optimal solution around the 56th iteration, and the optimal path length is 74.1, which is 32.64% more than the proposed algorithm. The BAS3 and ACO algorithms converge to the optimal solution around the 50th and 48th generation, with optimal paths of 67.6 and 64.2, which are 18.73% and 13.57% longer than that of our method. The optimal paths are 67.6 and 64.2, respectively, which are 18.73% and 13.57% more than that of our method. Compared with the SSA, both algorithms exhibit fewer turns; however, their ability to traverse ascending and descending obstacles is weaker. The proposed IABCBAS algorithm converges to the optimal solution at the 42nd iteration, with a shorter planned path, superior obstacle-traversing capability, and the smoothest and most rational path in this environment.

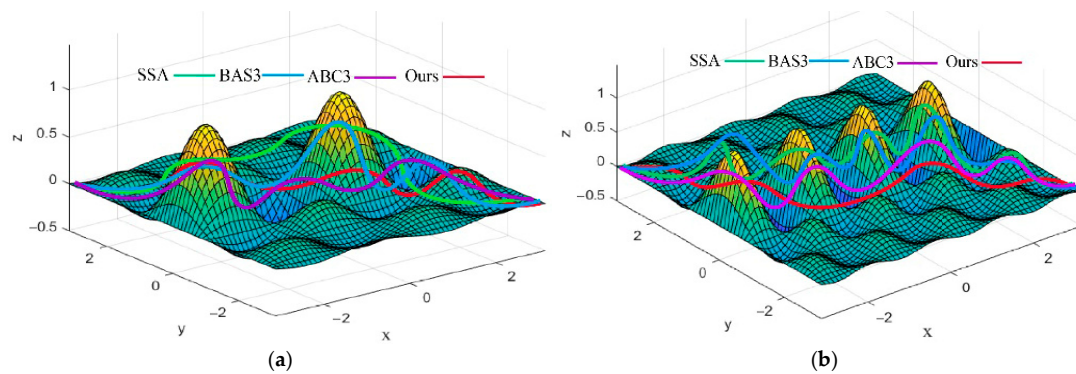


Figure 10. Optimal path planning results of four algorithms in the 3D environment: (a) twin peaks path planning results; (b) multimodal path planning results.

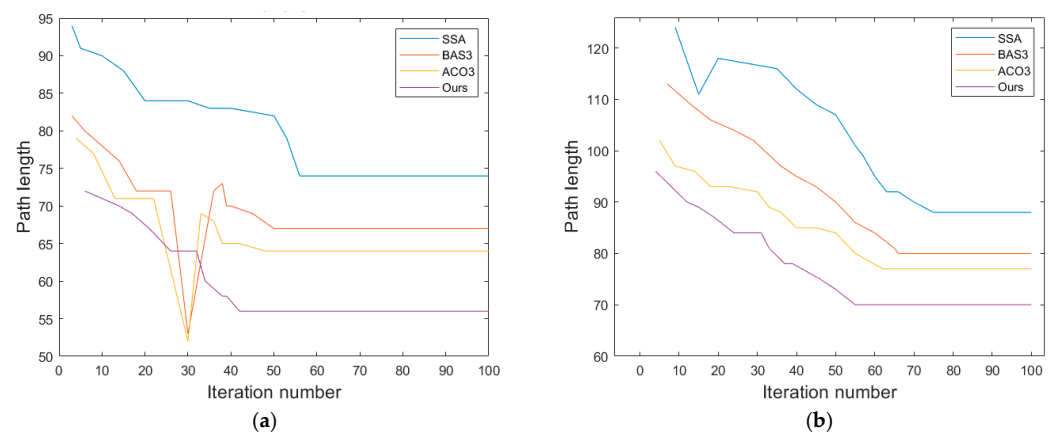


Figure 11. Convergence of 4 algorithmic paths in 3D environments: (a) convergence results for twin peaks environment; (b) convergence results for multimodal environment.

As shown in Table 6 and Figure 10, for path planning in 3D multimodal complex environment maps, the number of iterations required for convergence increases with the number of obstacles for the SSA, BAS3, and ABC3 algorithms, which converge to the optimal solution around the 75th, 66th, and 62nd iterations, respectively. Additionally, their paths exhibit more turns, increased length, and reduced obstacle-traversing capability. In contrast, the path generated by the proposed IACBAS algorithm is overall smooth. Compared with SSA, BAS3, and ABC3, the number of turns is reduced by 40.48%, 33.74%, and 26.65%, respectively. Moreover, the planning time for the shortest path is 13.62 s faster than the average of the three algorithms. These results indicate that the proposed IACBAS algorithm maintains superior planning performance in complex environments, meeting the current requirements of real-world applications.

Table 6. Comparison of average performance of four algorithms in three-dimensional environments.

Environments	Method	Average Path Length (m)	Average Number of Turns	Average Number of Iterations	Average Planning Time (s)
Three-dimensional unimodal simplex environment	SSA	77.2	35.2	58.5	52.76
	BAS3	69.1	30.1	52.6	44.36
	ACO3	66.1	27.4	50.6	41.38
	IACBAS	58.2	21.5	44.8	36.37
Three-dimensional multimodal complex environments	SSA	91.1	54.1	77.4	68.12
	BAS3	82.4	48.6	68.2	58.91
	ACO3	79.4	43.9	65.7	55.85
	IACBAS	72.1	32.2	57.2	47.34

4.5. Comparison Experiment of Real Environment Simulation

An experimental site was selected for the path planning test of the mobile platform—specifically, a real environment containing obstacles—and the site environment was modeled. The model was invoked according to experimental requirements, and the real site and its environment modeling results are presented in Figure 12. In Figure 12a, the white obstacles in the target vehicle test site are represented by black square grids in the model. Additionally, the partitions in the target vehicle test site correspond to the grid boundary lines in the modeled map shown in Figure 12b.

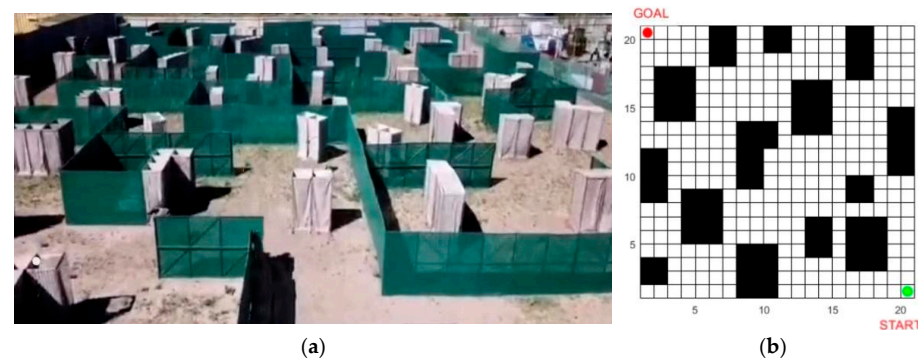


Figure 12. Real experimental environment and modeling: (a) real test environment; (b) real environment modeling.

The mobile platform calls the algorithm to perform autonomous path planning driving in the test site, and the corresponding planned path is generated on the map by calling the modeling map during driving. The effect of each algorithm on path planning in the real environment is shown in Table 7 and Figure 13.

Table 7. Comparison of average performance of four algorithms in real environments.

Environments	Method	Average Path Length (m)	Average Number of Turns	Average Number of Iterations	Average Planning Time (s)
Real environment	SSA	108.3	65.9	90.3	51.27
	BAS3	102.7	60.1	84.7	42.41
	ACO3	98.5	57.4	80.5	40.38
	IABCBAS	92.6	50.8	72.1	32.34

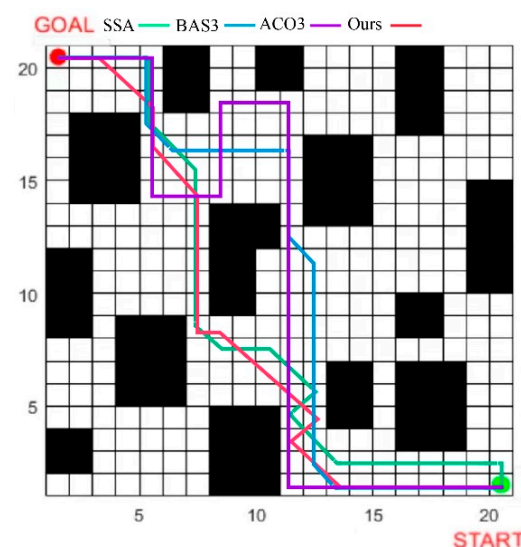


Figure 13. Route planning test results.

As presented in Table 7, our algorithm traverses the fewest grid squares in the simulation map and exhibits the optimal number of turns. The actual path distance is 92.6 m, which is 14.29%, 25.00%, and 35.71% shorter than those of the ACO3 (98.5 m), BAS3 (102.7 m), and SSA (108.3 m) algorithms, respectively. The planning time is 32.34 s, which is 16.14%, 24.86%, 31.14%, and 58.53% shorter than those of the four comparative algorithms, respectively. The proposed IABCBAS algorithm exhibits high efficiency in path planning for mobile target vehicles in real-world environments.

5. Conclusions

The IABCBAS algorithm is proposed to address the convergence and optimal search performance issues in swarm intelligence algorithms. Its search capability and stability are verified through tests on nine benchmark functions, with comparisons against four swarm intelligence algorithms from different periods. Ablation experiments are conducted to compare seven swarm intelligence algorithms with different optimization mechanisms, verifying the effectiveness of the introduced strategies in the improved algorithm. Application simulation results demonstrate that the proposed algorithm can rapidly plan a feasible path in the shortest time under environments of different dimensions. The proposed improvement mechanisms provide references for the optimization and enhancement of swarm intelligence algorithms, as well as their applications in other engineering fields. In future work, path planning can be integrated with curve processing strategies to further enhance path smoothness and planning accuracy. Additionally, the curvature optimization of inflection points can be improved to adapt to more complex real-world terrain environments.

Author Contributions: Conceptualization, S.W. and Y.D.; methodology, S.W. and M.Y.; software, S.W. and M.Y.; validation, S.W. and M.Y.; formal analysis, S.W. and Y.Z.; resources, S.W. and Y.D.; data curation, S.W. and Y.Z.; writing—original draft preparation, S.W. and Y.Z.; writing—review and editing, S.W. and Y.Z.; supervision, S.W. and Y.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the key research projects within the military grant number LJ20202A070636 and LJ20191A030128, and by the Tianjin Science and Technology Plan project grant number 22YFYSHZ00040. And The APC was funded by 22YFYSHZ00040.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding authors.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Liu, L.; Wang, X.; Yang, X.; Liu, H.; Li, J.; Wang, P. Path planning techniques for mobile robots: Review and prospect. *Expert Syst. Appl.* **2023**, *227*, 120254. [\[CrossRef\]](#)
2. Pan, J.; Lv, J.; Yan, L.; Weng, S.; Chu, S.; Xue, J. Golden eagle optimizer with double learning strategies for 3D path planning of UAV in power inspection. *Math. Comput. Simul.* **2022**, *193*, 509–532. [\[CrossRef\]](#)
3. Yao, P.; Qiu, L.; Qi, J.; Yang, R. AUV path planning for coverage search of static target in ocean environment. *Ocean Eng.* **2021**, *241*, 110050. [\[CrossRef\]](#)
4. Beasley, J. A discrete optimisation approach for target path planning whilst evading sensors. *Comput. Oper. Res.* **2022**, *138*, 105587. [\[CrossRef\]](#)
5. Cheng, C.; Sha, Q.; He, B.; Li, G. Path planning and obstacle avoidance for AUV: A review. *Ocean Eng.* **2021**, *235*, 109355. [\[CrossRef\]](#)

6. Leonardo, S.; Flavio, T. DVG plus A* and RRT Path-Planners: A Comparison in a Highly Dynamic Environment. *J. Intell. Robot. Syst.* **2021**, *101*, 58.
7. Li, H.; Zhao, T.; Dian, S. Forward search optimization and subgoal-based hybrid path planning to shorten and smooth global path for mobile robots. *Knowledge-Based Syst.* **2022**, *258*, 110034. [\[CrossRef\]](#)
8. Zhao, L.; Bai, Y.; Paik, K. Global-local hierarchical path planning scheme for unmanned surface vehicles under dynamically unforeseen environments. *Ocean Eng.* **2023**, *280*, 114750. [\[CrossRef\]](#)
9. Aljaleud, F.; Kurdi, H. Youcef-Toumi. Bio-Inspired Multi-UAV Path Planning Heuristics: A Review. *Mathematics* **2023**, *11*, 2356. [\[CrossRef\]](#)
10. Sui, F.; Tang, X.; Dong, Z.; Gan, X.; Luo, P.; Sun, J. ACO+PSO+A*: A bi-layer hybrid algorithm for multi-task path planning of an AUV. *Comput. Ind. Eng.* **2023**, *175*, 108905. [\[CrossRef\]](#)
11. Zhao, W.; Du, C.; Jiang, S. An adaptive multiscale approach for identifying multiple flaws based on XFEM and a discrete artificial fish swarm algorithm. *Comput. Methods Appl. Mech. Eng.* **2018**, *339*, 341–357. [\[CrossRef\]](#)
12. Sabitri, P.; Muhammad, A.; Sangman, M. Bio-Inspired Optimization-Based Path Planning Algorithms in Unmanned Aerial Vehicles: A Survey. *Sensors* **2023**, *23*, 3051.
13. Chen, X.; Tang, J.; Lao, S. Review of unmanned aerial vehicle swarm communication architectures and routing protocols. *Appl. Sci.* **2020**, *10*, 3661. [\[CrossRef\]](#)
14. Bakirci, M. A novel swarm unmanned aerial vehicle system: Incorporating autonomous flight, real-time object detection, and coordinated intelligence for enhanced performance. *Trait. Signal* **2023**, *40*, 2063–2078. [\[CrossRef\]](#)
15. Cui, Y.; Hu, W.; Ahmed, R. Multi-robot path planning using learning-based artificial bee colony algorithm. *Eng. Appl. Artif. Intell.* **2023**, *129*, 107579. [\[CrossRef\]](#)
16. Ni, X.; Hu, W.; Fan, Q.; Cui, Y.; Qi, C. A Q-learning based multi-strategy integrated artificial bee colony algorithm with application in unmanned vehicle path planning. *Expert Syst. Appl.* **2023**, *236*, 121303. [\[CrossRef\]](#)
17. Li, G.; Liu, C.; Wu, L.; Xiao, W. A mixing algorithm of ACO and ABC for solving path planning of mobile robot. *Appl. Soft Comput.* **2023**, *148*, 110868. [\[CrossRef\]](#)
18. Serap-Ercan, C.; Harun-Resit, Y. A new approach based on hybrid ant colony optimization-artificial bee colony algorithm for multi-objective electric vehicle routing problems. *Eng. Appl. Artif. Intell.* **2023**, *123*, 106375.
19. Liang, Q.; Zhou, H.; Yin, Y.; Xiong, W. An improved beetle antennae search path planning algorithm for vehicles. *PLoS ONE* **2022**, *17*, e0274646. [\[CrossRef\]](#)
20. Yu, Z.; Yuan, J.; Li, Y.; Yuan, C.; Deng, S. A path planning algorithm for mobile robot based on water flow potential field method and beetle antennae search algorithm. *Comput. Electr. Eng.* **2023**, *109*, 108730. [\[CrossRef\]](#)
21. Chen, Y.; Li, J.; Zhang, L. Learning sparrow algorithm with non-uniform search for global optimization. *Int. J. Swarm Intell. Res.* **2023**, *14*, 31. [\[CrossRef\]](#)
22. Geng, J.; Sun, X.; Wang, H.; Bu, X.; Liu, D.; Li, F.; Zhao, Z. A modified adaptive sparrow search algorithm based on chaotic reverse learning and spiral search for global optimization. *Neural Comput. Appl.* **2023**, *35*, 24603–24620. [\[CrossRef\]](#)
23. Sarada, M.; Prabhujit, M. Fast random opposition-based learning Golden Jackal Optimization algorithm. *Knowl.-Based Syst.* **2023**, *275*, 110679.
24. Hu, G.; Huang, F.; Chen, K.; Wei, G. MNEARO: A meta swarm intelligence optimization algorithm for engineering applications. *Comput. Methods Appl. Mech. Eng.* **2023**, *419*, 116664. [\[CrossRef\]](#)
25. Mohsen, R.; Matthias, F. No-Free-Lunch Theorems for Reliability Analysis. *Asce-Asme J. Risk Uncertain. Eng. Syst. Part A-Civ. Eng.* **2023**, *9*, 04023019.
26. Wang, Y.; Zhang, X.; Yu, D.; Bai, Y.; Du, J.; Tian, Z. Tent Chaotic Map and Population Classification Evolution Strategy-Based Dragonfly Algorithm for Global Optimization. *Math. Probl. Eng.* **2022**, *2022*, 2508414. [\[CrossRef\]](#)
27. Jiang, C.; Yang, S.; Nie, P.; Xiang, X. Multi-objective structural profile optimization of ships based on improved Artificial Bee Colony algorithm and structural component library. *Ocean Eng.* **2023**, *283*, 115124. [\[CrossRef\]](#)
28. Laith, A.; Mohamed, E.; Putra, S.; Zong-Woo, G.; Amir, G. Reptile search algorithm(RSA):A nature-inspired meta-heuristic optimizer. *Expert Syst. Appl.* **2022**, *191*, 116158.
29. Xiao, W.; Li, G.; Liu, C.; Tan, L. A novel chaotic and neighborhood search-based artificial bee colony algorithm for solving optimization problems. *Sci. Rep.* **2023**, *13*, 20496. [\[CrossRef\]](#)
30. Chen, H.; Liang, Y.; Meng, X. A UAV Path Planning Method for Building Surface Information Acquisition Utilizing Opposition-Based Learning Artificial Bee Colony Algorithm. *Remote Sens.* **2023**, *15*, 4312. [\[CrossRef\]](#)
31. Aveen-Othman, A.; Daniel, P.; Shafi, G.; Ali, G. The Application of Hybrid Krill Herd Artificial Hummingbird Algorithm for Scientific Workflow Scheduling in Fog Computing. *J. Bionic Eng.* **2023**, *20*, 2443–2464. [\[CrossRef\]](#)
32. Zhang, Y.; Li, T.; Li, Z.; Wu, Y.; Miao, H. Software Defects Prediction Based on Hybrid Beetle Antennae Search Algorithm and Artificial Bee Colony Algorithm with Comparison. *Axioms* **2022**, *11*, 305. [\[CrossRef\]](#)

33. Dong, E.; Chen, Z.; Wang, R.; Zhao, J. Optimization of Two-Dimensional Extended Warranty Scheme for Failure Dependence of a Multi-Component System with Improved PSO-BAS Algorithm. *Processes* **2022**, *10*, 1479. [\[CrossRef\]](#)
34. Ni, J.; Tang, J.; Wang, R. Hybrid Algorithm of Improved Beetle Antenna Search and Artificial Fish Swarm. *Appl. Sci.* **2022**, *12*, 13044. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.